SOUTENANCE PROJET

Système d'acquisition / adaptation / conversion des signaux d'un recepteur de modelisme - Projet 32

SOMMAIRE

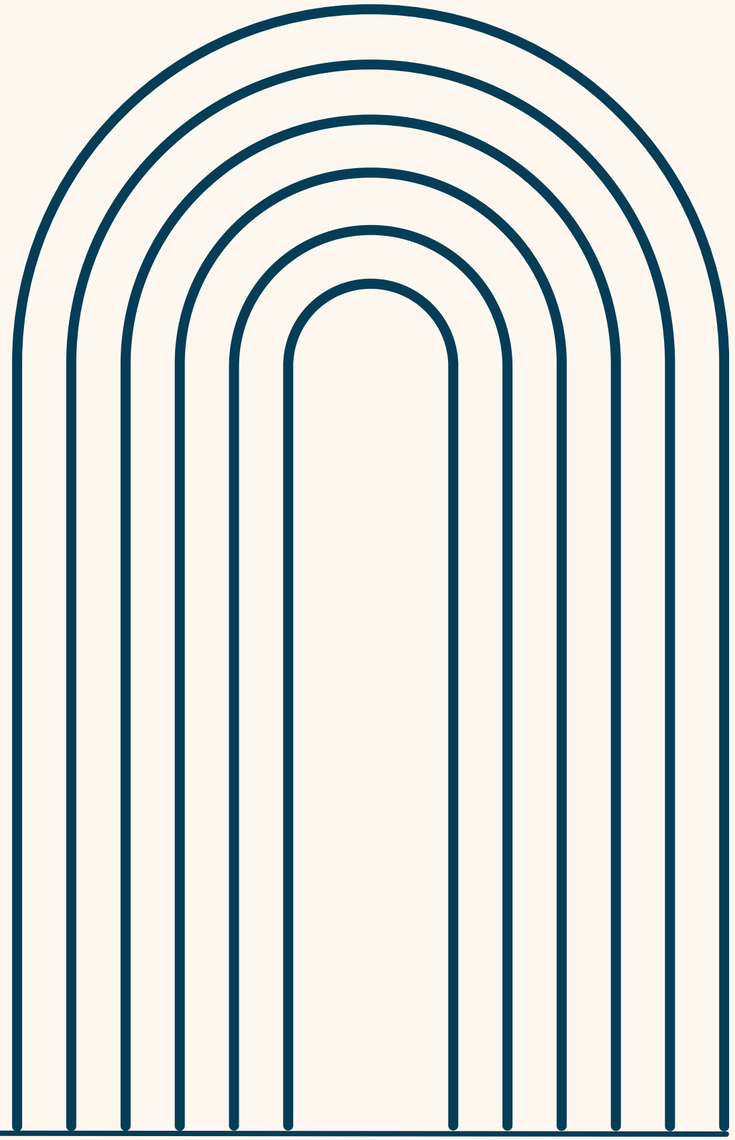
01 Notre équipe

02 Présentation de la Partie I

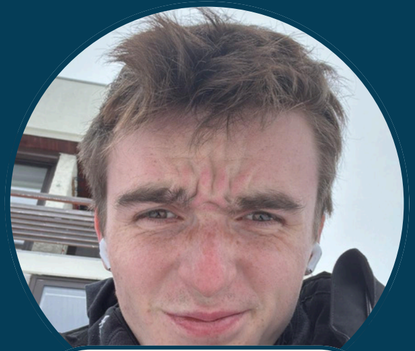
03 Présentation de la Partie II

04 Tests finaux de la Partie II

05 Utilisation de l’IA



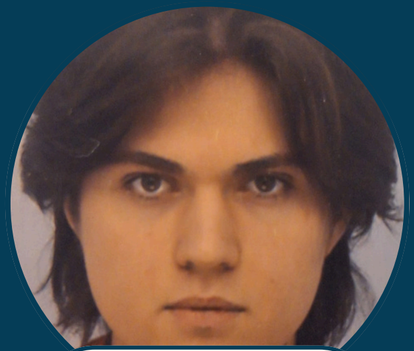
Notre équipe
Partie I



Martin



Mathis



Jules



Simon

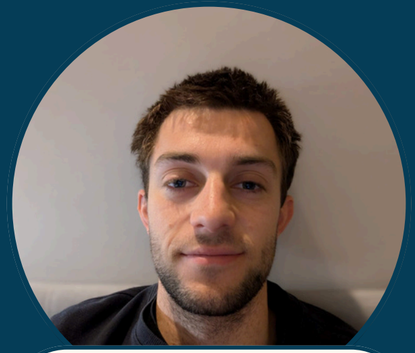
Notre équipe
Partie II



Aurélien



Corentin P



Corentin E



Amaury



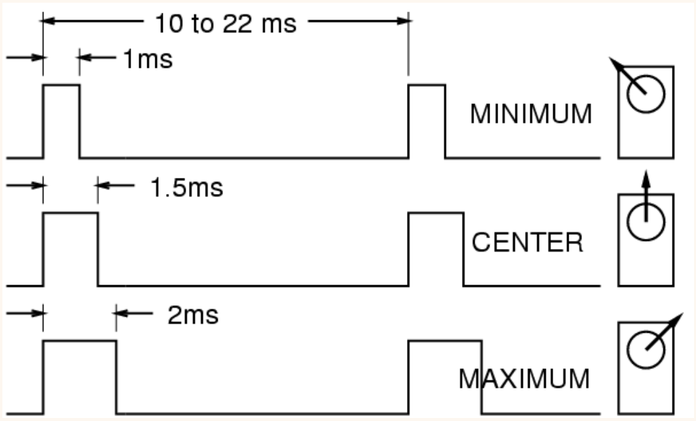
Reda

PRÉSENTATION DE LA PARTIE I

MARTIN / MATHIS / JULES / SIMON

CONTEXTE

Problème : Protocole PWM	Solution : Protocole S.Bus
• 1 fil physique par voie de commande	• 16 voies sur 1 seul fil — standard Futaba
• Fréquence 50 Hz, impulsions 1 000–2 000 µs	• 100 000 bauds, format 8E2, logique inversée
• Câblage complexe pour systèmes multi-actionneurs	• Trame de 25 octets à 111 Hz



CAHIER DES CHARGES

Interface d'Adaptation PWM

Acquisition :

Lire 6 signaux PWM indépendants (E1 à E6) en entrée. Précision de mesure à la microseconde.

Redistribution :

Retransmettre chaque signal d'entrée vers une sortie miroir dédiée (S1 à S6) avec un délai imperceptible.

Encodage numérique :

Générer une trame S.Bus de 16 voies à 100 000 bauds sur un unique fil de sortie, conforme au standard Futaba.

Contraintes matérielles :

Fonctionnement sur Raspberry Pi Pico (RP2040). Pas de système d'exploitation. Temps réel garanti par interruptions GPIO et PWM matériel.

POINTS CLÉS DU CODE :

$$v = v_{\min} + \frac{\tau - \tau_{\min}}{\tau_{\max} - \tau_{\min}} \times (v_{\max} - v_{\min})$$

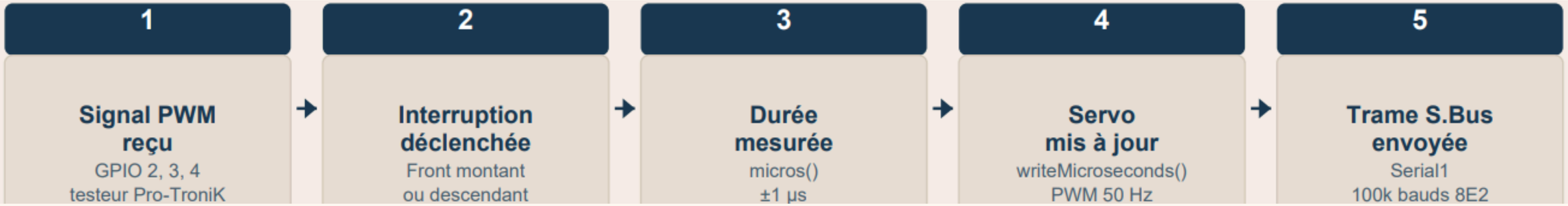
Interruptions GPIO : attachInterrupt() déclenché sur chaque changement d'état (CHANGE). Mesure de la durée impulsion via micros(). Filtrage des valeurs hors plage [1000 ; 2000] µs.

Une callback par voie : Arduino n'autorise pas de paramètres dans les callbacks d'interruption. Une fonction dédiée par voie (callback_voie_0, callback_voie_1, callback_voie_2) contourne cette contrainte.

Sorties servo : Bibliothèque Servo.h avec writeMicroseconds() pour piloter directement en µs. Signal PWM 50 Hz généré en hardware, sans charge CPU.

Encodage S.Bus : Conversion linéaire PWM→11 bits par voie, empilement bit à bit sur 25 octets, inversion logique ~trame[i] et envoi Serial1 à 100 000 bauds 8E2.

EXPLICATION DU CODE :




```
1   // Front montant : démarre le chrono
2   if (digitalRead(ENTREES[0]) == HIGH)
3       debut_impulsion[0] = micros();
4   // Front descendant : mesure + filtre
5   else {
6       duree = micros() - debut_impulsion[0];
7       if (duree >= 1000 && duree <= 2000)
8           valeurs_pwm[0] = duree;
9   }
```

Callback (le point le plus important)

ARCHITECTURE DE L'EXPÉRIENCE

Testeur Pro-TroniK

3 sorties PWM



→

Raspberry Pi Pico (RP2040)

Acquisition
Redistribution
Encodage S.Bus

Sortie S.Bus
100 000 bauds · 8E2 · 16 voies



→

Servomoteurs × 3

PWM 50 Hz



RÉSULTATS ARDUINO

Valeur minimale :

```
V1=1004µs  V2=1004µs  V3=1004µs  |  Trame: F0 4D 6F 7A D3 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
```

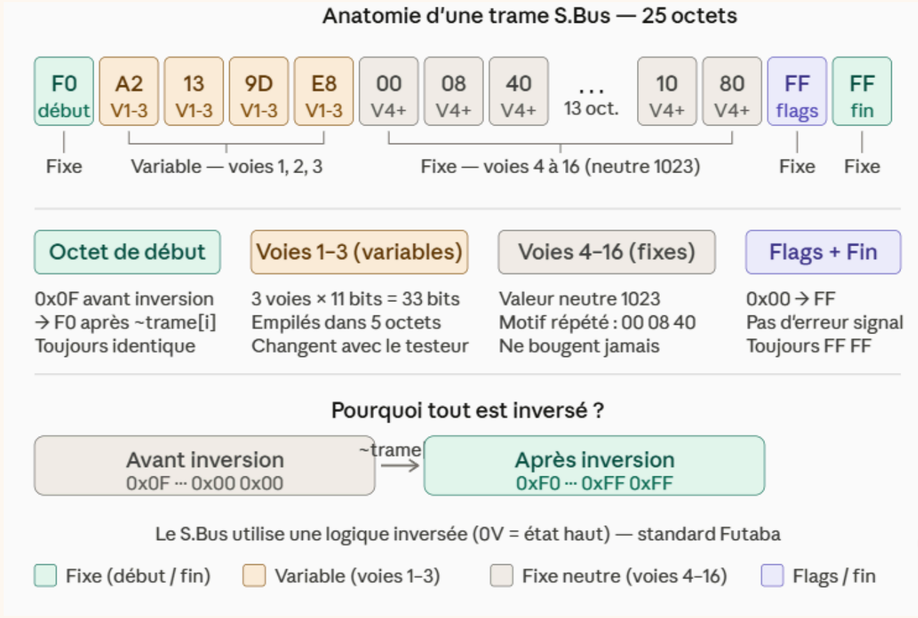
Valeur neutre :

```
V1=1506µs  V2=1506µs  V3=1506µs  |  Trame: F0 16 B4 A0 05 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
```

Valeur maximale :

```
V1=1991µs  V2=1991µs  V3=1991µs  |  Trame: F0 FB D8 C7 3E 00 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
```

Étape	Action / Opération	Illustration Binaire / Hexa	Valeur Décimale
1. Poids faible (8 bits)	Lecture de l'Octet 1	0x5D	**93**
2. Poids fort (3 bits)	Extraction sur l'Octet 2	1110 1100 → on garde 100	**4**
3. Alignement	Décalage à gauche (<< 8)	Multiplication par 28 (4* 256)	**1024**
4. Résultat Voie 1	Fusion des deux parties		**93 + 1024 = 1117**



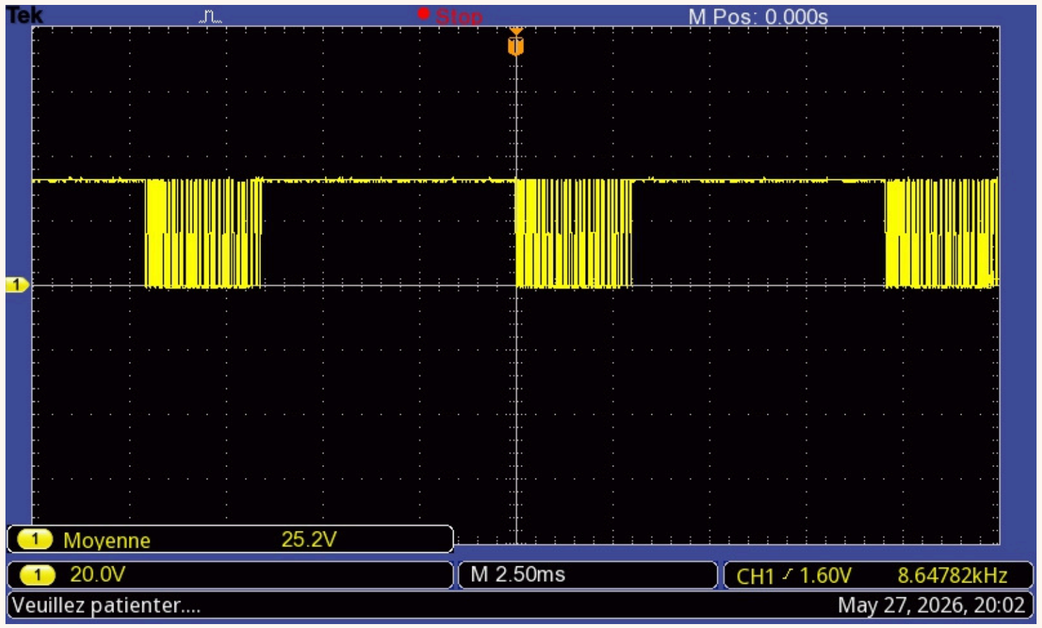
VÉRIFICATION DES RÉSULTATS

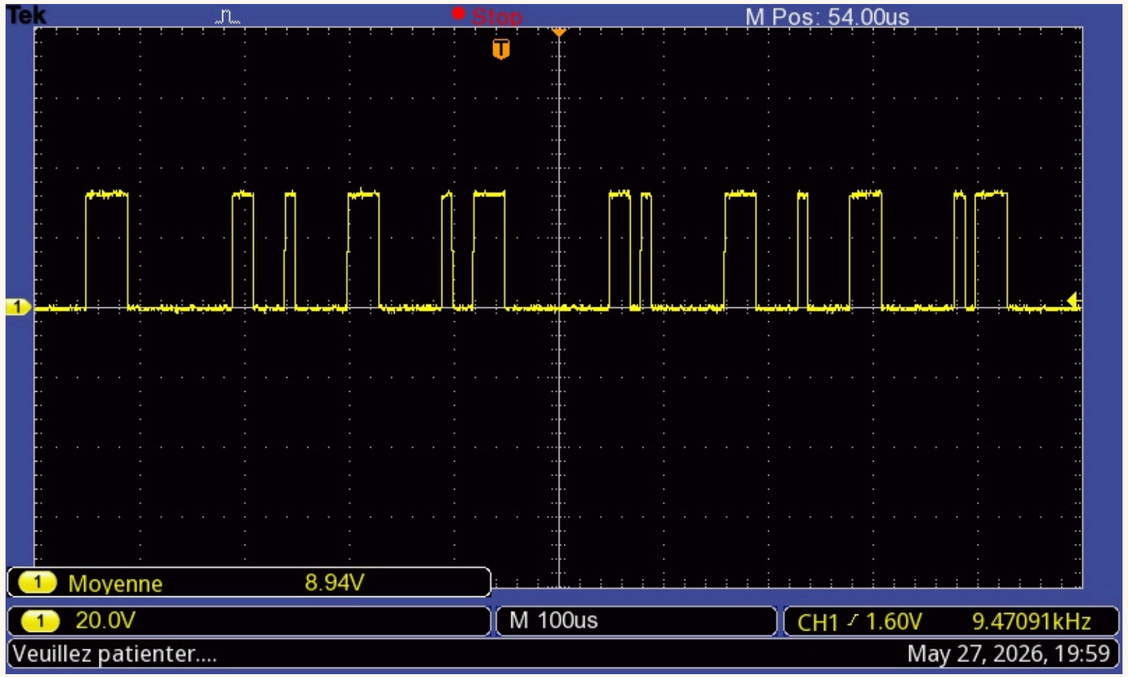
Points clés :

- Base de temps oscilloscope : 2,50 ms / division
- Observations physiques :
  - Espacement entre deux trames : environ 3,6 divisions
  - Largeur d'un bloc de données : environ 1,2 divisions

Preuves mathématiques :

- Période de rafraîchissement mesurée : 3,6 div × 2,5 ms/div = 9 ms (Conforme au standard)
- Temps de transmission théorique (25 octets, format 8E2 = 12 bits/octet) : (25*12)/100000 = 3ms (Correspond à la largeur d'1,2 division mesurée à l'écran)



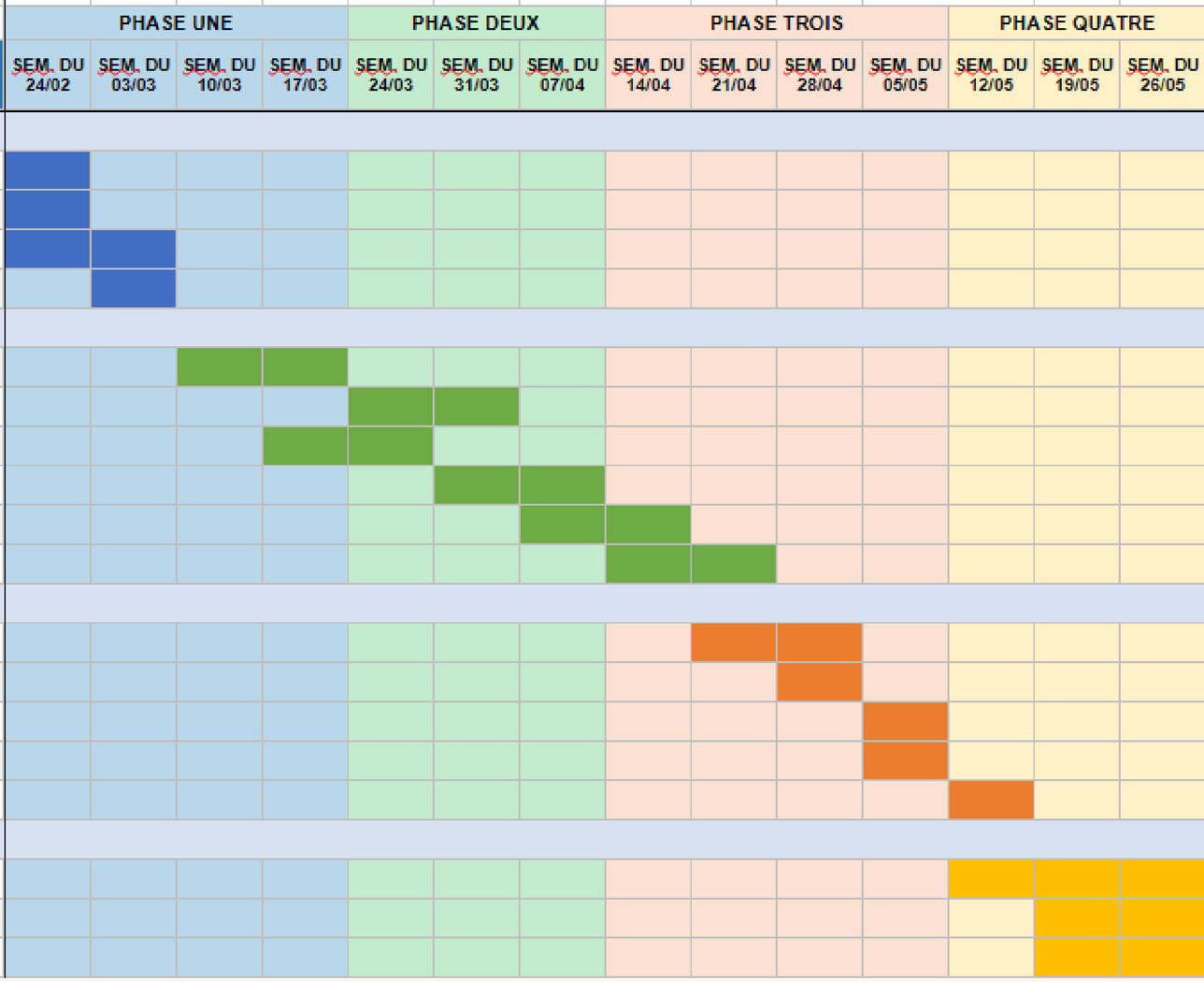


Points clés :
Base de temps oscilloscope : 100 µs / division
Configuration matérielle (Pico) : UART S.Bus à 100 000 bauds
Preuves mathématiques :
Durée d'un bit sur la ligne : 1/100000 = 10 µs
Durée totale d'un octet S.Bus :
12 bits × 10 µs = 120 µs
L'encodage matériel du microcontrôleur est parfaitement aligné sur la norme.

DIAGRAMME DE GANTT

TITRE	Adaptateur PWM → S.Bus		Numéro du PR	32		
CHEF	Martin FLEURY		DATE	24/02/2026		

N°	TITRE DE LA TÂCHE	PROPRIÉTAIRE	DATE DE DÉBUT	DATE LIMITE	DURÉE (EN J)	TÂCHE TERMINÉE (%)
1	**Découverte et cadrage du projet**					
1.1	Prise en main du sujet et du matériel	Tous	24/02/2026	03/03/2026	7	100 %
1.2	Définition du cahier des charges	Tous	24/02/2026	03/03/2026	7	100 %
1.3	Recherche sur les protocoles PWM/S.Bus	Jules / Simon	24/02/2026	10/03/2026	14	100 %
1.4	Choix de l'architecture matérielle	Mathis / Simon	03/03/2026	10/03/2026	7	100 %
2	**Développement logiciel**					
2.1	Code Python (acquisition PWM + mock)	Martin / Mathis	10/03/2026	24/03/2026	14	100 %
2.2	Code Python simplifié (sans mock)	Martin	24/03/2026	07/04/2026	14	100 %
2.3	Portage C Raspberry Pi (pigpio)	Simon	17/03/2026	31/03/2026	14	100 %
2.4	Génération signal CPPM (C + Python)	Mathis / Jules	31/03/2026	14/04/2026	14	100 %
2.5	Génération signal S.Bus (C + Python)	Martin / Simon	07/04/2026	21/04/2026	14	100 %
2.6	Code C Raspberry Pi Pico (pico-sdk)	Martin / Jules	14/04/2026	28/04/2026	14	100 %
3	**Intégration et tests matériels**					
3.1	Adaptation code Arduino IDE (Pico)	Mathis	21/04/2026	05/05/2026	14	100 %
3.2	Test acquisition PWM (testeur Pro-TroniK)	Mathis / Simon	28/04/2026	05/05/2026	7	100 %
3.3	Test redistribution servo (Dualsky DS3001)	Jules / Simon	05/05/2026	12/05/2026	7	100 %
3.4	Test encodage S.Bus + moniteur série	Martin / Jules	05/05/2026	12/05/2026	7	100 %
3.5	Validation trame S.Bus à l'oscilloscope	Martin	12/05/2026	19/05/2026	7	100 %
4	**Finalisation et soutenance**					
4.1	Rédaction rapport LaTeX	Martin	12/05/2026	27/05/2026	15	100 %
4.2	Création diapositives soutenance	Tous	19/05/2026	27/05/2026	8	100 %
4.3	Préparation soutenance	Tous	24/05/2026	27/05/2026	3	80 %



CONCLUSION PARTIE 1 :

CE QUI A ÉTÉ FAIT :

Acquisition PWM :
3 voies lues par interruptions GPIO, précision ±1 µs, filtrage 1000–2000 µs

Redistribution miroir :
3 servos pilotés en temps réel, latence < 20 ms, PWM 50 Hz matériel

Encodage S.Bus :
trame 25 octets à 100 000 bauds, validée à l'oscilloscope

PISTES D´AMÉLIORATIONS :

Entrée CPPM :
remplacer 3 fils PWM par 1 seul fil multiplexé depuis le récepteur RC

Test récepteur RC réel :
valider avec un vrai récepteur au lieu du testeur Pro-TroniK

PRÉSENTATION DE LA PARTIE II

AURÉLIEN / CORENTIN P / CORENTIN E / AMAURY / REDA

CAHIER DES CHARGES

CONCEVOIR ET DÉPLOYER UN SYSTÈME EMBARQUÉ AUTONOME CAPABLE DE SE SUBSTITUER À LA TÉLÉCOMMANDE D'ORIGINE POUR FORCER L'EXTINCTION DES VIDÉOPROJECTEURS LORSQUE LA SALLE DES CONSEILS N'EST PLUS UTILISÉE, SANS NÉCESSITER D'INTERVENTION HUMAINE.

Détection de présence :

Le système doit s'assurer que la salle est vide pour effectuer l'extinction.

Discrétion du système :

L'installation dissimulée pour ne pas dénaturer la Salle des Conseils et éviter les dégradations involontaires.

Condition horaire :

Extinction forcée et systématique déclenchée tous les jours à 21h00 (tous le temps en week-end).

Autonomie :

Capacité à conserver l'heure exacte même en cas de coupure de courant dans le bâtiment.

DIAGRAMME D'EXIGENCE

- Tiroir 1
- Tiroir 2
- Tiroir 3

29/05/26

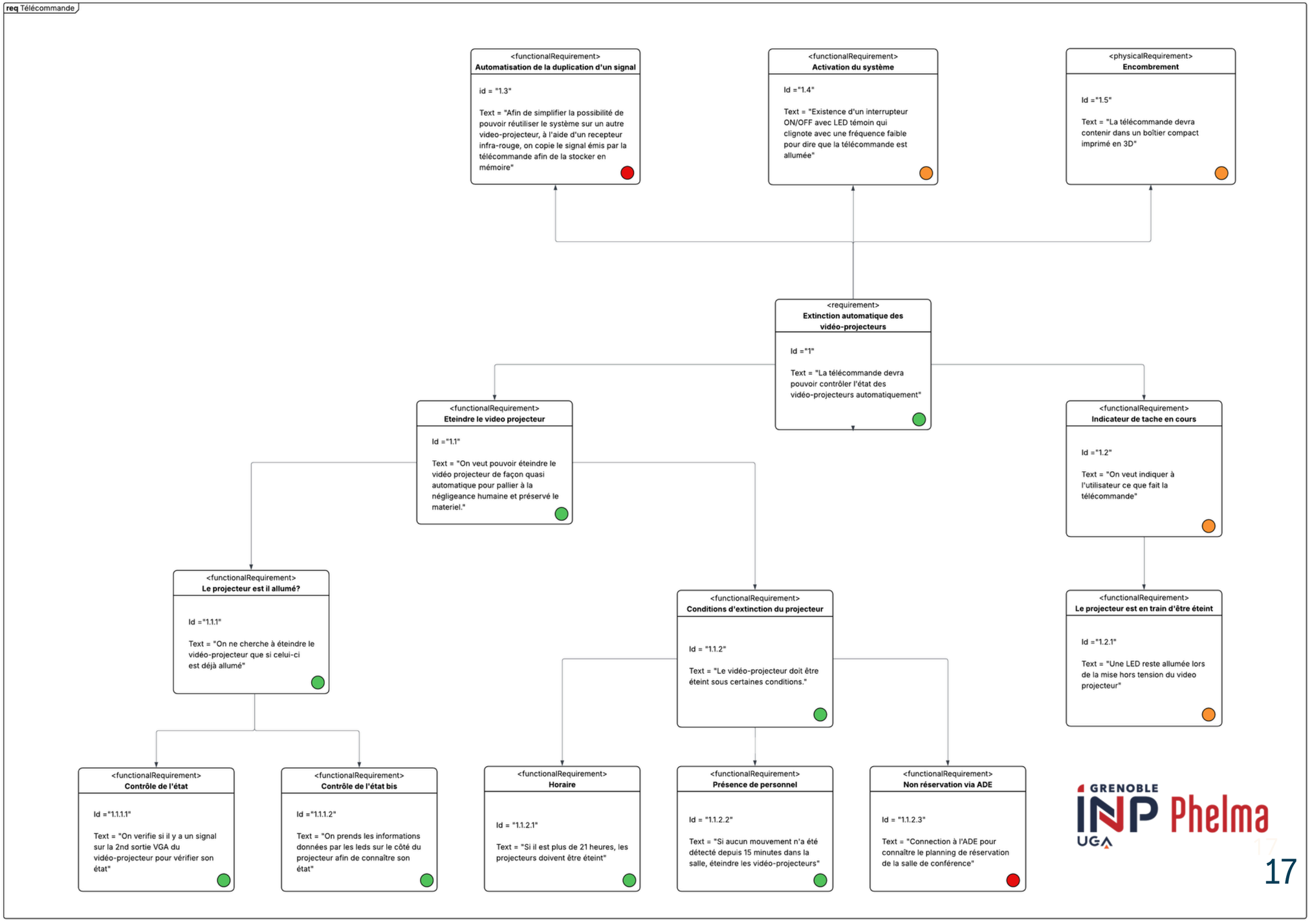
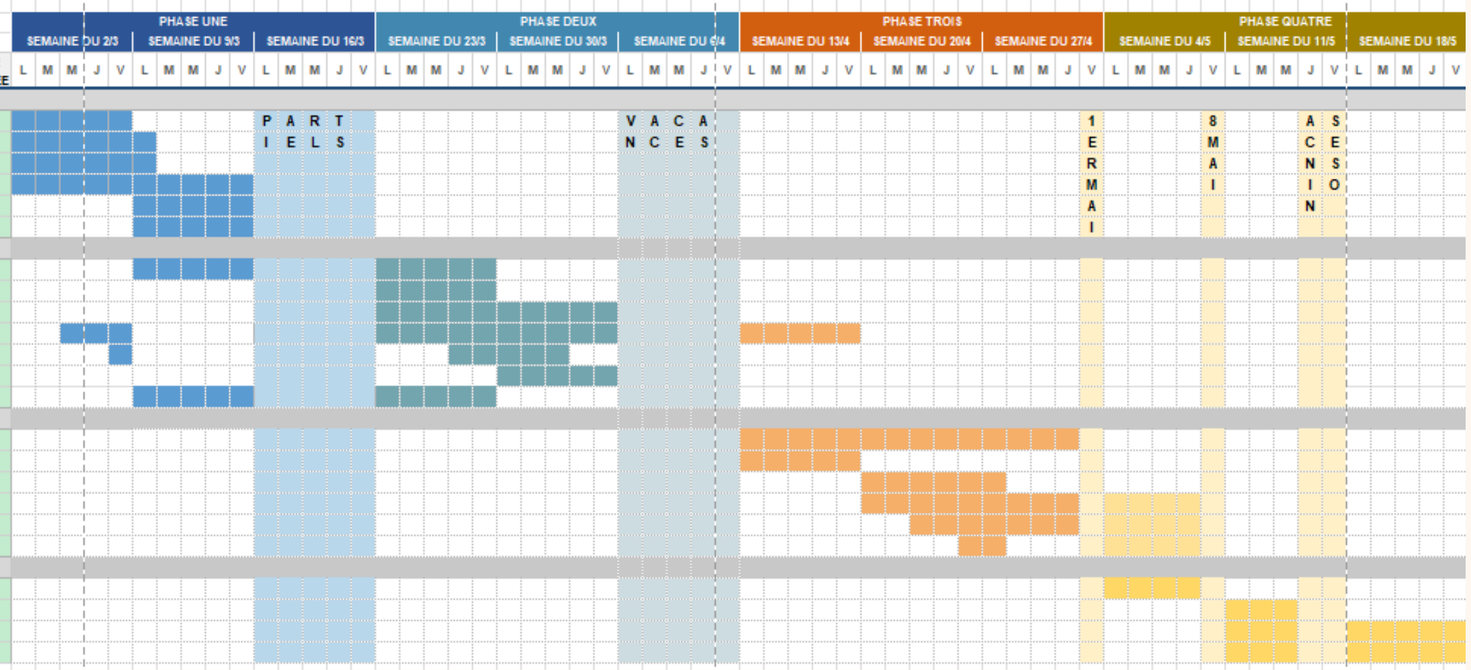


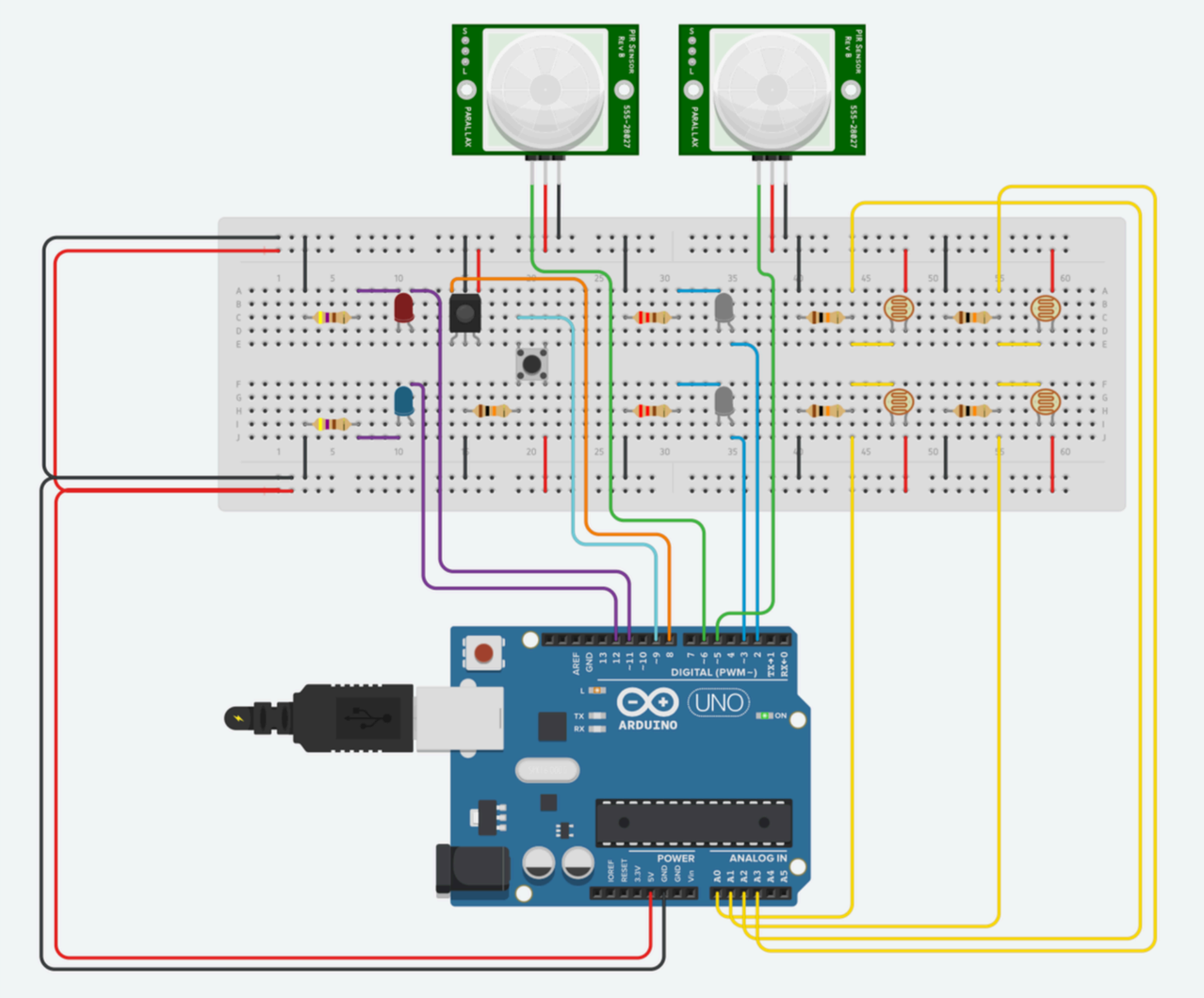
DIAGRAMME DE GANTT

TITRE DU PROJET	Télécommande pour vidéo-projecteur	Numéro du Projet	32
CHEF DE PROJET	Aurélien	DATE	02/03/2026

NUMÉRO	TITRE DE LA TÂCHE	PROPRIÉTAIRE DE LA TÂCHE	DATE DE DÉBUT	DATE LIMITE	DURÉE	TÂCHE TERMINÉE
1	**Découverte environnement du projet**					
1.1	Prise de connaissance du projet / premier contact avec le chef de projet	Tous	02/03/26	06/03/26	4j	100%
1.2	Définition de la cible du projet et du cahier des charges	Tous	02/03/26	06/03/26	4j	100%
1.3	Recherche du plan d'attaque pour répondre au cahier des charges	Tous	02/03/26	08/03/26	6j	100%
1.4	Création du diagramme des exigences	Aurélien, Corentin P., Reda	03/03/26	09/03/26	6j	100%
1.5	Budgétisation de la première commande	Corentin E.	02/03/26	09/03/26	7j	100%
1.6	Mise en route du projet	Tous	03/03/26	13/03/26	10j	100%
2	**Conception de la télécommande**					
2.1	Modèle de la télécommande	Amaury	24/03/26	28/03/26	4j	100%
2.2	Saisie des schémas sur le logiciel 3D	Corentin P.	23/03/26	03/04/26	4j	100%
2.3	Conception de la structure de la télécommande	Tous	23/03/26	03/04/26	8j	100%
2.4	Écriture du code pour VGA	Aurélien	02/03/26	03/04/26	8j	100%
2.5	Écriture du code pour infrarouge	Aurélien	02/03/26	03/04/26	4j	100%
2.6	Choix des modules d'information (leds)	Tous	03/03/26	04/04/26	4j	100%
2.7	Implémentation module capteur de mouvement	Aurélien, Reda, Corentin P.	03/03/26	27/03/26	8j	100%
3	**Assemblage et tests**					
3.1	Assemblage de la télécommande	À définir	13/04/26	01/05/26	12j	100%
3.2	Test fonctionnement leds	Amaury	13/04/26	01/05/26	4j	100%
3.2.1	Test fonctionnement VGA	Corentin E.	13/04/26	01/05/26	4j	100%
3.2.2	Test envoie de commande infrarouge	Reda	20/04/26	08/05/26	8j	100%
3.3	Test système entier	Amaury	20/04/26	08/05/26	4j	100%
3.3.1	Mise en place de la télécommande dans la salle de conseil	Aurélien	23/04/26	08/05/26	2j	100%
4	**Finalisation/contrôle du projet**					
4.1	Derniers ajustements	Tous	04/05/26	08/05/26	4j	100%
4.2	Vérification conformité objectifs du projet	Tous	11/05/26	15/05/26	4j	100%
4.3	Rédaction du rendu	Tous	11/05/26	22/05/26	12j	100%
4.4	Préparation soutenance	Tous	11/05/26	22/05/26	12j	100%



CIRCUIT ÉLECTRONIQUE



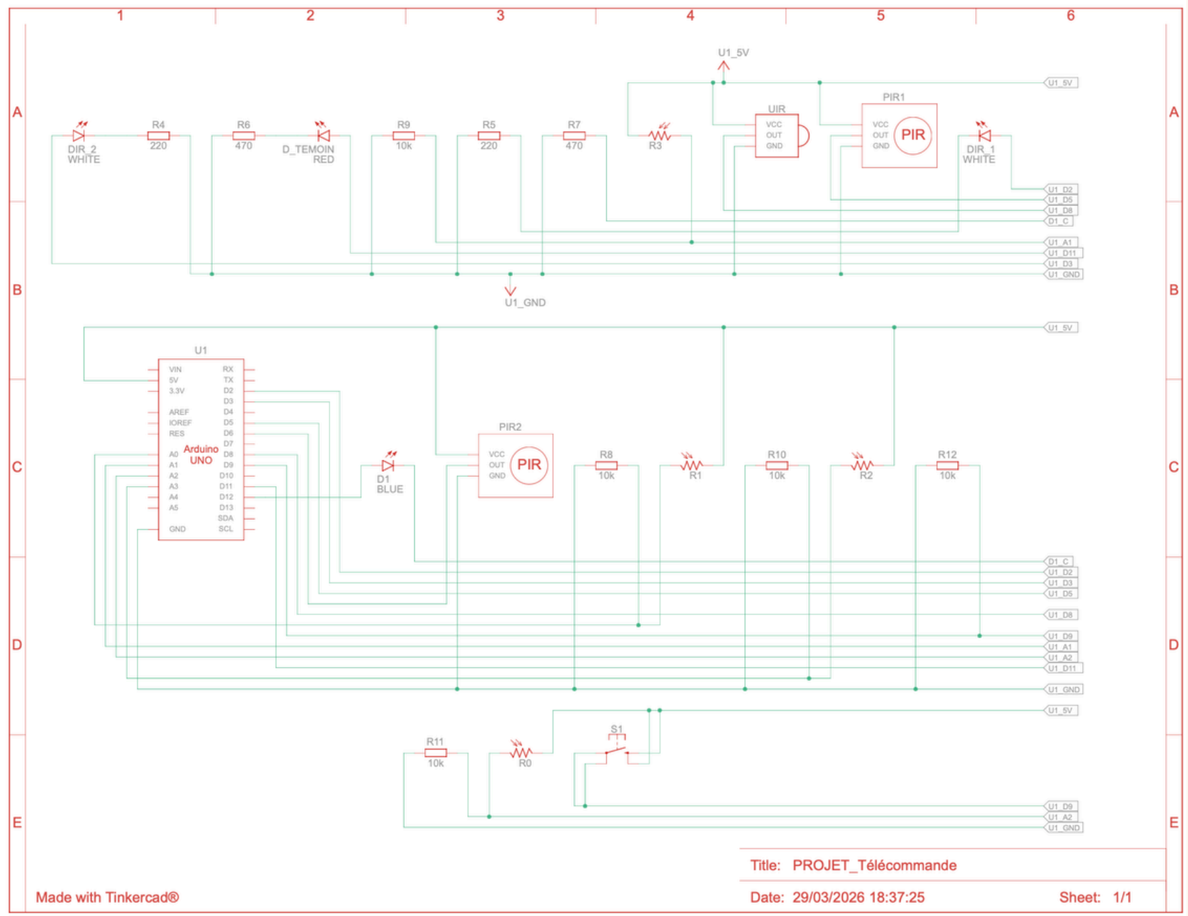


SCHÉMA ÉLECTRIQUE

TABLEAU BUDGÉTAIRE

Materiel GoTronic : *

Composant	Code GOTronic	Quantité	Prix Unitaire TTC	Total
Carte Raspberry Pi Pico 2WH SC1634	39295	1	10.95	10.95€
Détecteur sonore SEN-14262	39461	1	16.80	16.80€
Détecteur de présence mmWave Gravity SEN0609	39143	1	16.70	16.70€
Détecteur de mouvement PIR SBC-PIR	37072	1	6.90	6.90€
Photorésistance PGM1201	03136	4	1.80	7.20€
Module emetteur IR GT1087	35228	2	2.4	4.8€
Module recepteur IR GT1089	35229	1	2.4	2.4€
Bouton-poussoir R1381R	07099	1	1.5	1.5€
Total				**67.25€**

DÉGRADATION DU MATÉRIEL

12H/NUIT ~2X PAR SEMAINE 1250H/AN

prix d'une lampe : ~100€ - 200€
vie totale de la lampe : 3000h-4000h
→ 33% de la lampe consumé sans intérêt

ACQUISITION DE L'ÉTAT DU VIDÉO-PROJECTEUR

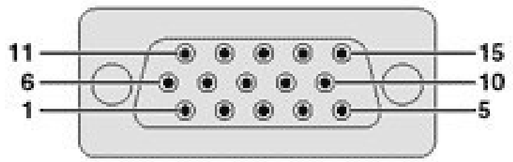
```arduino
#define LED 2
#define videoProjecteur1 A2
#define videoProjecteur2 A3

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  if (analogRead(videoProjecteur1) > 512 || analogRead(videoProjecteur2) > 512) {
    digitalWrite(LED, HIGH);
  } else {
    digitalWrite(LED, LOW);
  }
}
```

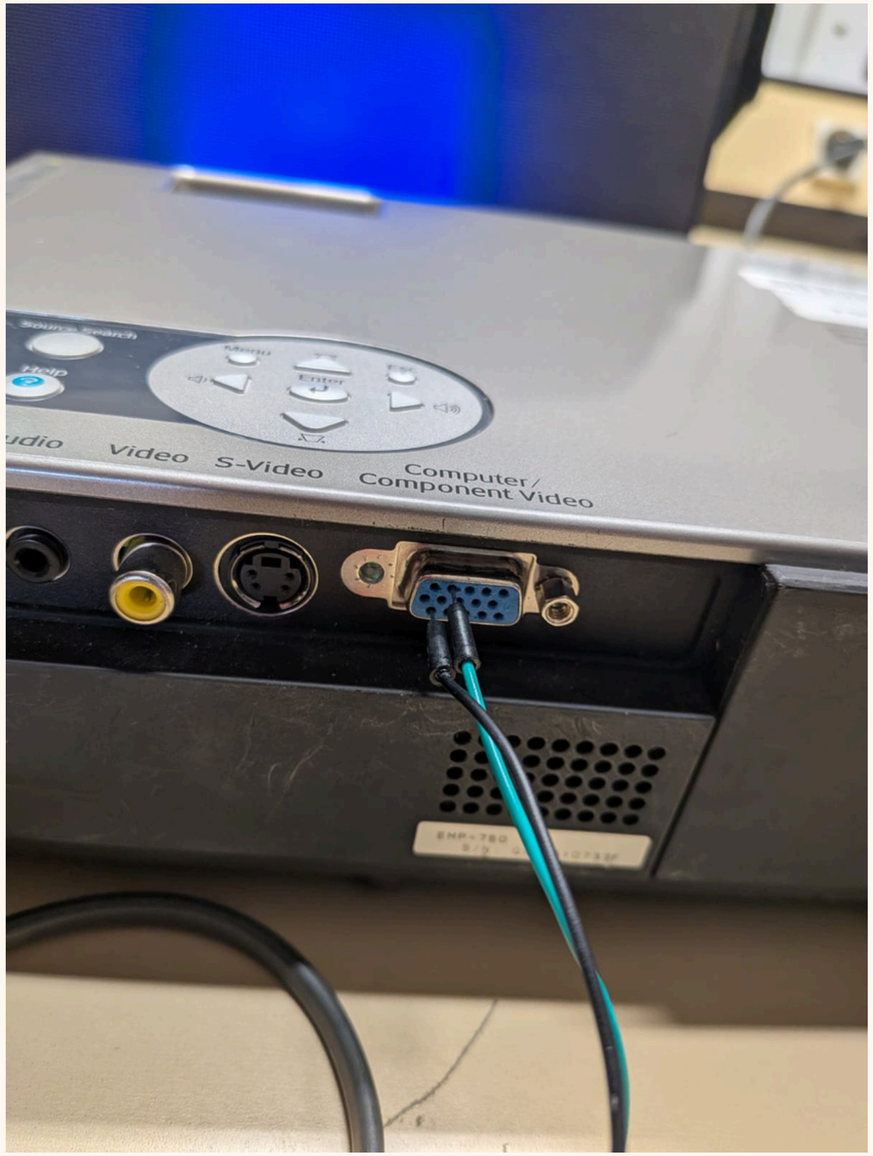
CODE ARDUINO AVEC LED TÉMOIN POUR OBTENIR L'ÉTAT DES VIDÉO-PROJECTEURS

SCHÉMA 'CLASSIQUE' CONNECTEUR VGA



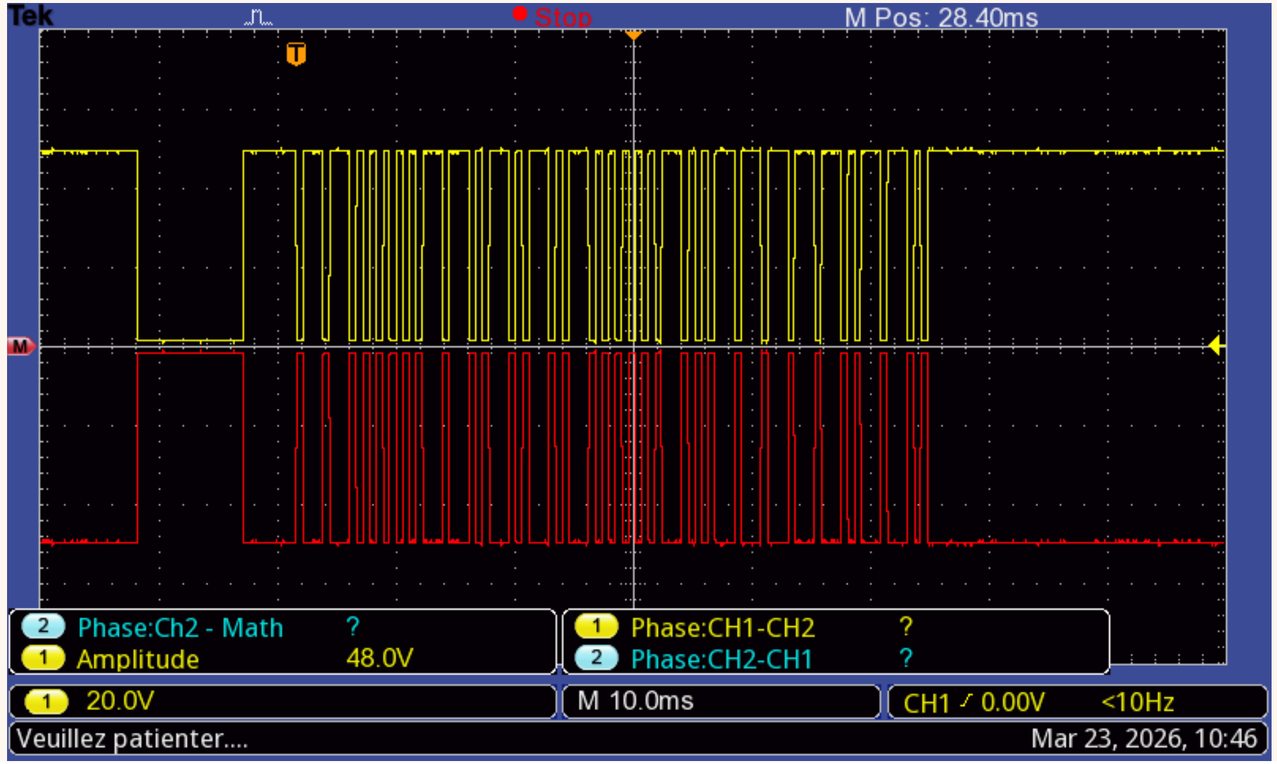
RGB Input
Analog

1. Video input (red)
2. Video input (green/sync on green)
3. Video input (blue)
4. Not connected
5. Composite sync
6. Earth (red)
7. Earth (green/sync on green)
8. Earth (blue)
9. Not connected
10. GND
11. GND
12. Bi-directional data
13. Horizontal sync signal
14. Vertical sync signal
15. Data clock

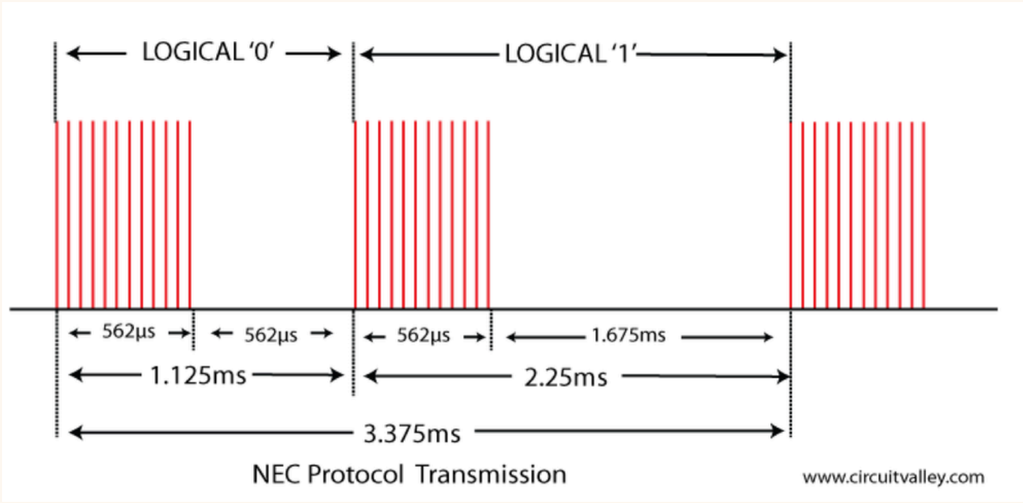


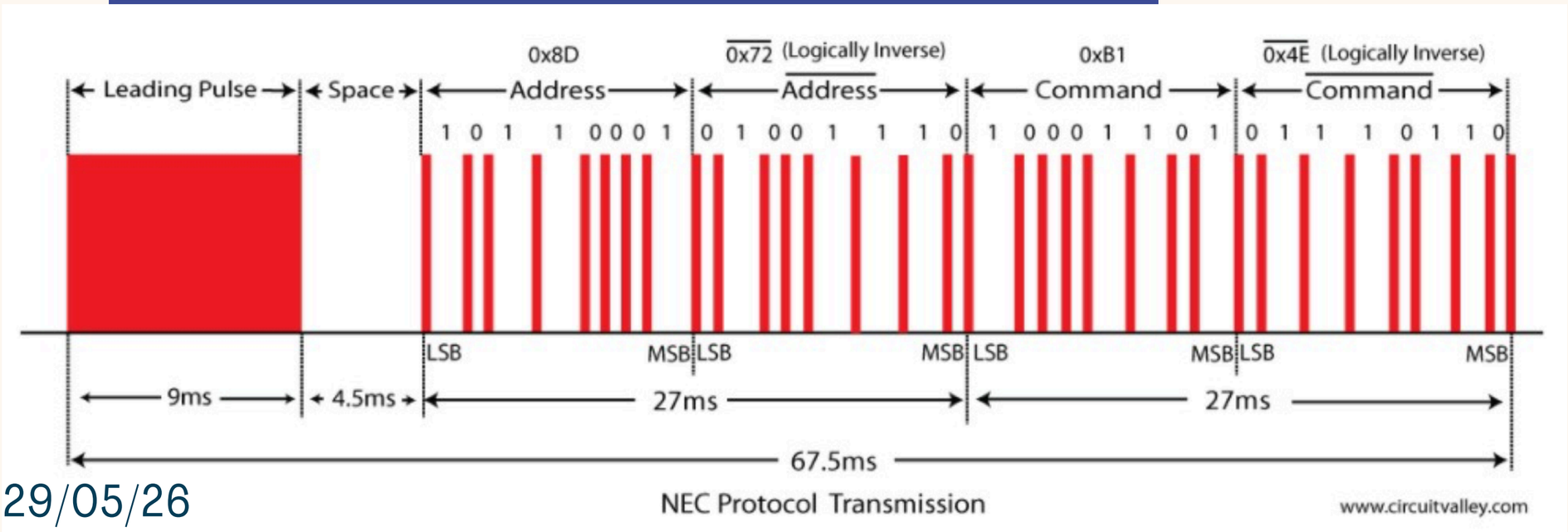
FIL VERT : GND
FIL NOIR : SORTIE VIDÉO

SIGNAL DE LA TÉLÉCOMMANDE



PROTOCOLE NEC





SIGNAL RETENU :

11000001
10101010
00001001
11110110

CONVERSION DU SIGNAL EN HEXADÉCIMAL

```arduino
#include <IRremote.h>

int RECV_PIN = 11;

void setup() {
  Serial.begin(9600);

  IrReceiver.begin(RECV_PIN, ENABLE_LED_FEEDBACK);

  Serial.println("Systeme pret. En attente du signal...");
}

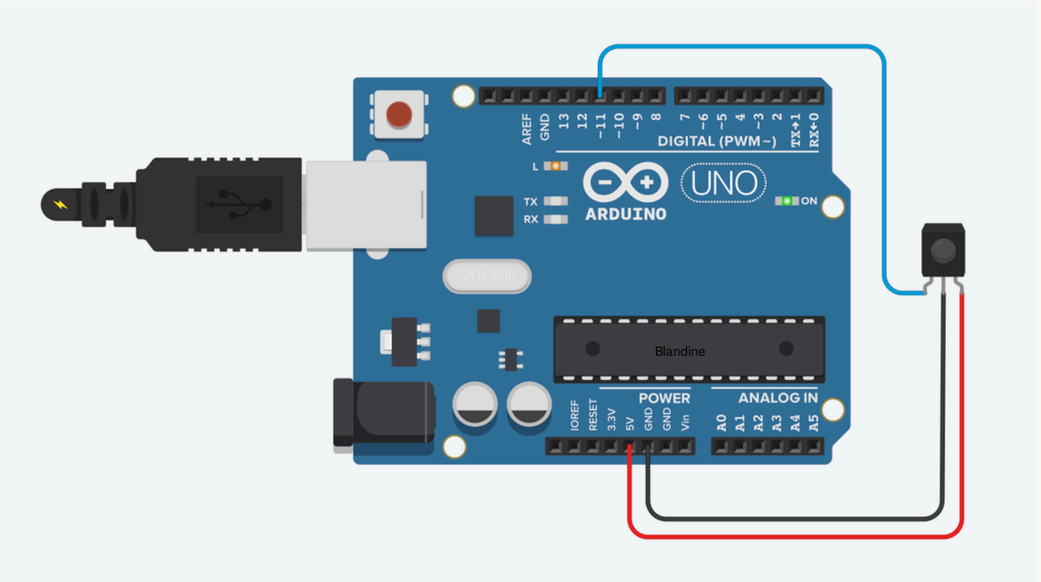
void loop() {

  if (IrReceiver.decode()) {

    // Affiche la valeur brute du signal en Hexadécimal
    Serial.println(IrReceiver.decodedIRData.decodedRawData, HEX);

    // Prépare le récepteur à écouter le prochain signal
    IrReceiver.resume();
  }
}
```

CODE ARDUINO RÉCEPTION IR*




```
21:31:30.713 -> Systeme pret. En attente du signal...
21:31:33.910 -> 6F905583
21:31:38.948 -> 6F905583
21:31:44.025 -> 6F905583
21:31:49.096 -> 6F905583
```

Signal retenu	1100	0001	1010	1010	0000	1001	1111	0110
Gauche à droite	C	1	A	A	0	9	F	6
Droite à gauche	3	8	5	5	0	9	F	6

SIGNAL HÉXADÉCIMAL RETENU : 6F905583

*CODE DISPONIBLE DANS LA DATASHEET DU RÉCEPTEUR IR SUR GOTRONIC

REPRODUCTION DU SIGNAL RETENU HEXADÉCIMAL

```ino
#include <IRremote.h>

const int IR_SEND_PIN = 3;

void setup() {
  Serial.begin(9600);

  IrSender.begin(IR_SEND_PIN);

  Serial.println("Système prêt ! Émission dans 3 secondes...");
  delay(3000);
}

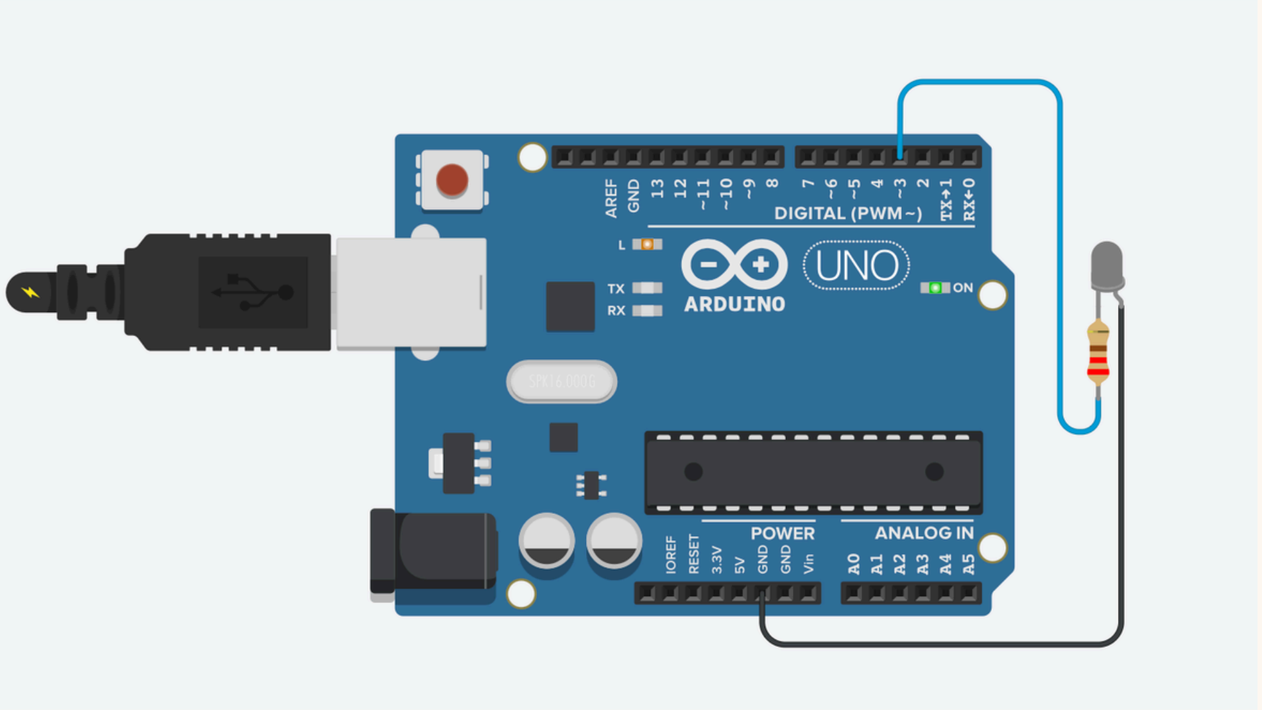
void loop() {
  Serial.println("Envoi du code infrarouge...");

  uint32_t monCode = 0x6F905583;

  IrSender.sendNECRaw(monCode, 0);

  delay(5000);
}
```

CODE ARDUINO ÉMISSION IR*




```
22:11:49.815 -> Système prêt ! Émission dans 3 secondes...
22:11:52.786 -> Envoi du code infrarouge...
22:11:57.867 -> Envoi du code infrarouge...
22:12:02.941 -> Envoi du code infrarouge...
22:12:07.987 -> Envoi du code infrarouge...
```

L'ENVOI DE CE SIGNAL IR FAIT ALLUMER/ÉTEINDRE LE VIDÉO-PROJECTEUR

*CODE EXEMPLE DISPONIBLE DANS L'IDE ARDUINO (FICHIER>EXEMPLES>IRREMOTE>SIMPLESENDER)

RECONSTRUCTION DES BITS NEC

```arduino
#define ledPin 4

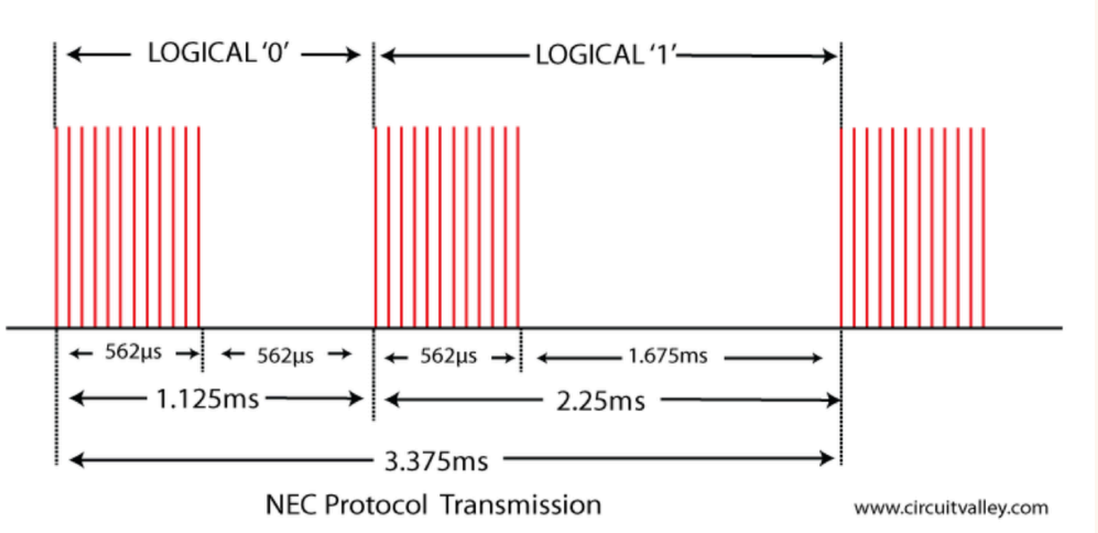
void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
  Serial.println("Systeme pret. Envoi manuel du code NEC.");
}

// ==========================================
// 1. GÉNÉRATION DES SIGNAUX (Couche Physique)
// ==========================================

// La fonction "mark" fait clignoter la LED à 38 kHz
void mark(unsigned int duree) {
  unsigned long debut = micros();

  // Tant que le temps imparti n'est pas écoulé, on clignote
  while (micros() - debut < duree) {
    digitalWrite(ledPin, HIGH);
    delayMicroseconds(13); // 13 microsecondes allumé
    digitalWrite(ledPin, LOW);
    delayMicroseconds(13); // 13 microsecondes éteint
  }
}

// La fonction "space" maintient la LED éteinte (silence)
void space(unsigned int duree) {
  digitalWrite(ledPin, LOW);
  delayMicroseconds(duree);
}
```

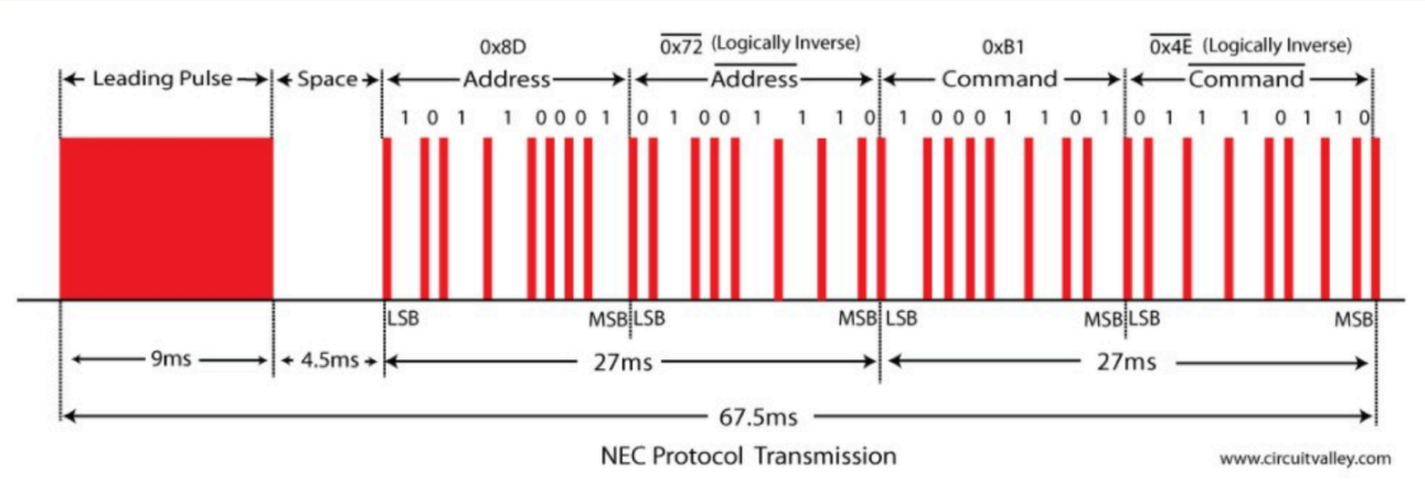



```arduino
// ==========================================
// 2. CONSTRUCTION DES BITS NEC
// ==========================================

// Un "0" logique : 562µs de signal, suivi de 562µs de silence
void zero() {
  mark(562);
  space(562);
}

// Un "1" logique : 562µs de signal, suivi de 1675µs de silence
void one() {
  mark(562);
  space(1675);
}
```

REPRODUCTION DU SIGNAL RETENU HEXADÉCIMAL




```
22:56:09.114 -> Systeme pret. Envoi manuel du code NEC.
22:56:09.114 -> Envoi du signal...
22:56:09.180 -> Signal envoye avec succes. Pause de 5 secondes.
```

```
23:02:22.719 -> Systeme pret. En attente du signal...
23:02:22.817 -> 6F905583
23:02:27.894 -> 6F905583
23:02:32.960 -> 6F905583
```

```cpp
// ==========================================
// 3. SÉQUENCE DE 32 BITS
// ==========================================

void code() {
  one(); one(); zero(); zero(); zero(); zero(); zero(); one();
  one(); zero(); one(); zero(); one(); zero(); one(); zero();
  zero(); zero(); zero(); zero(); one(); zero(); zero(); one();
  one(); one(); one(); one(); zero(); one(); one(); zero();
}

// ==========================================
// 4. BOUCLE PRINCIPALE
// ==========================================

void loop() {
  Serial.println("Envoi du signal...");

  // Le Header (Signal de départ du protocole NEC)
  mark(9000);   // Signal de 9 ms
  space(4500);  // Silence de 4.5 ms

  // Transmission des 32 bits de données
  code();

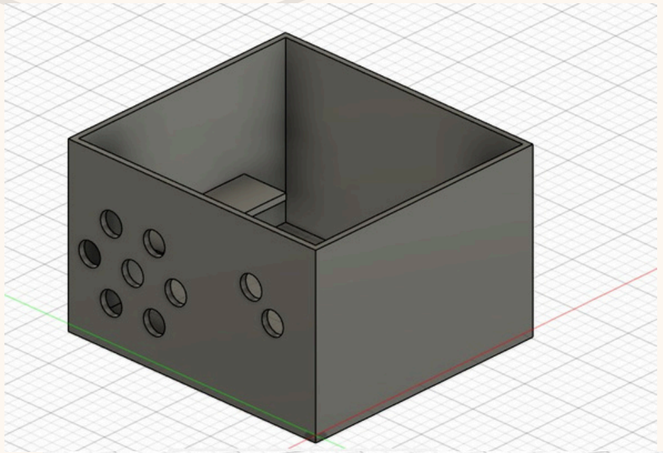
  // Fin de transmission et pause
  digitalWrite(ledPin, LOW); // On s'assure que la LED est bien éteinte
  Serial.println("Signal envoye avec succes. Pause de 5 secondes.");

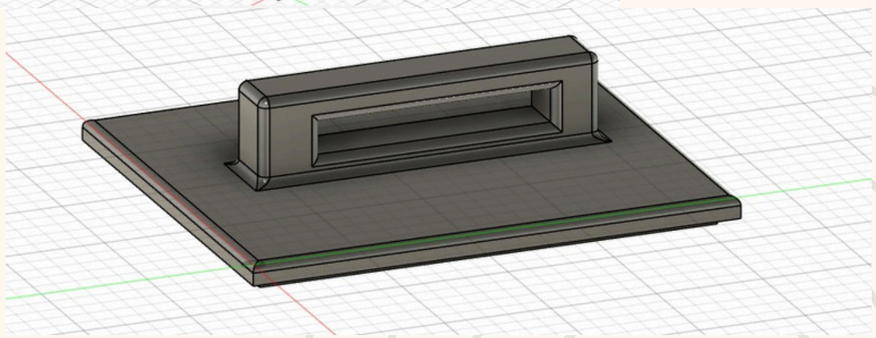
  delay(5000);
}
```

ON A RÉUSSI A REPRODUIRE LE SIGNAL ENVOYÉ PAR LA TÉLÉCOMMANDE INITIALE

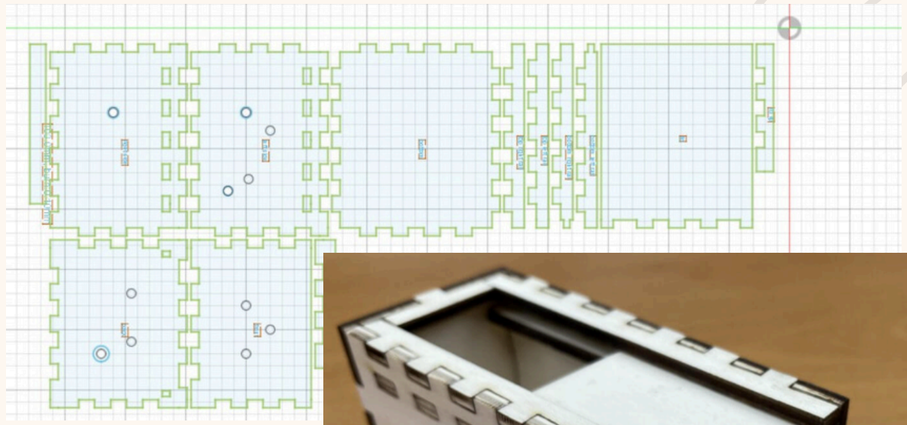
MISE EN FORME DU SYSTÈME

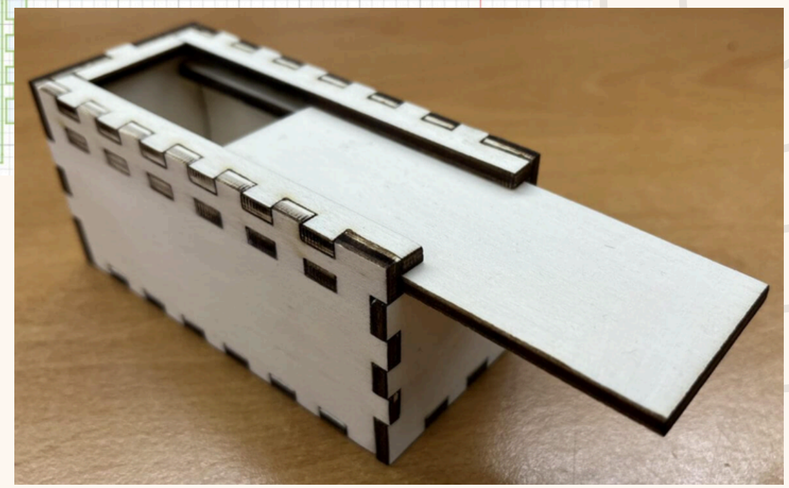
IDÉE N°1 :





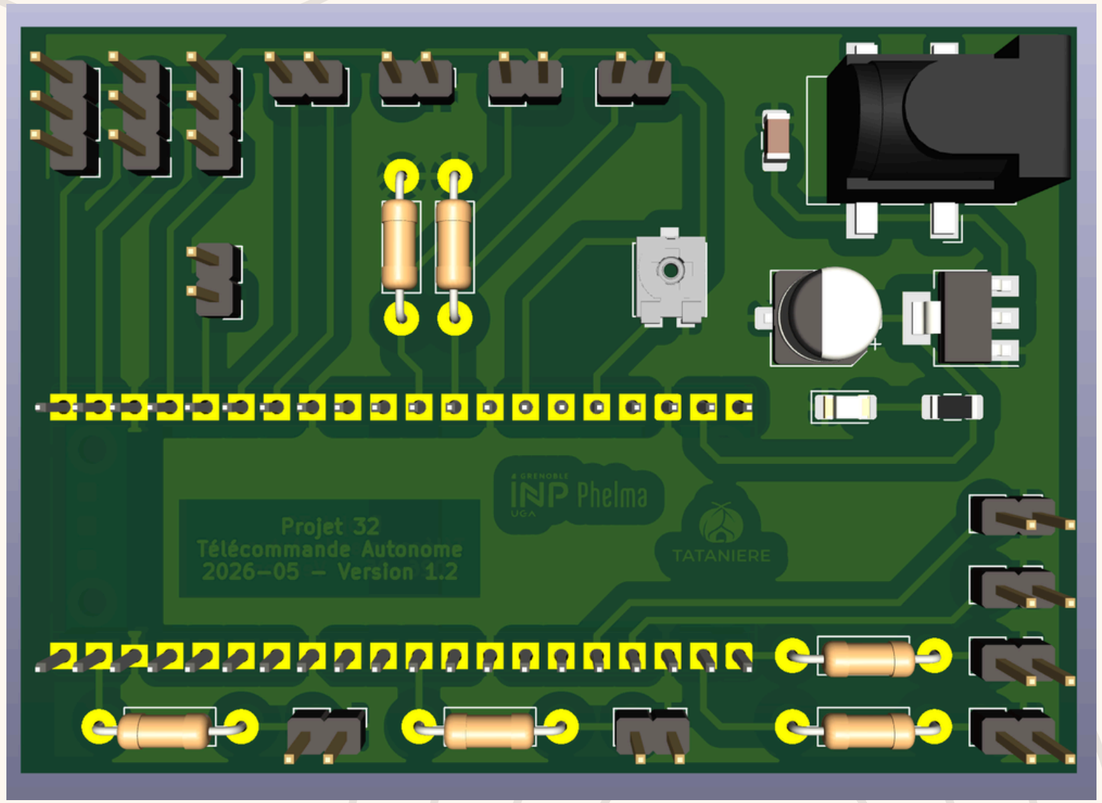
IDÉE N°2 :





MISE EN FORME DU SYSTÈME

PLAQUE PCB



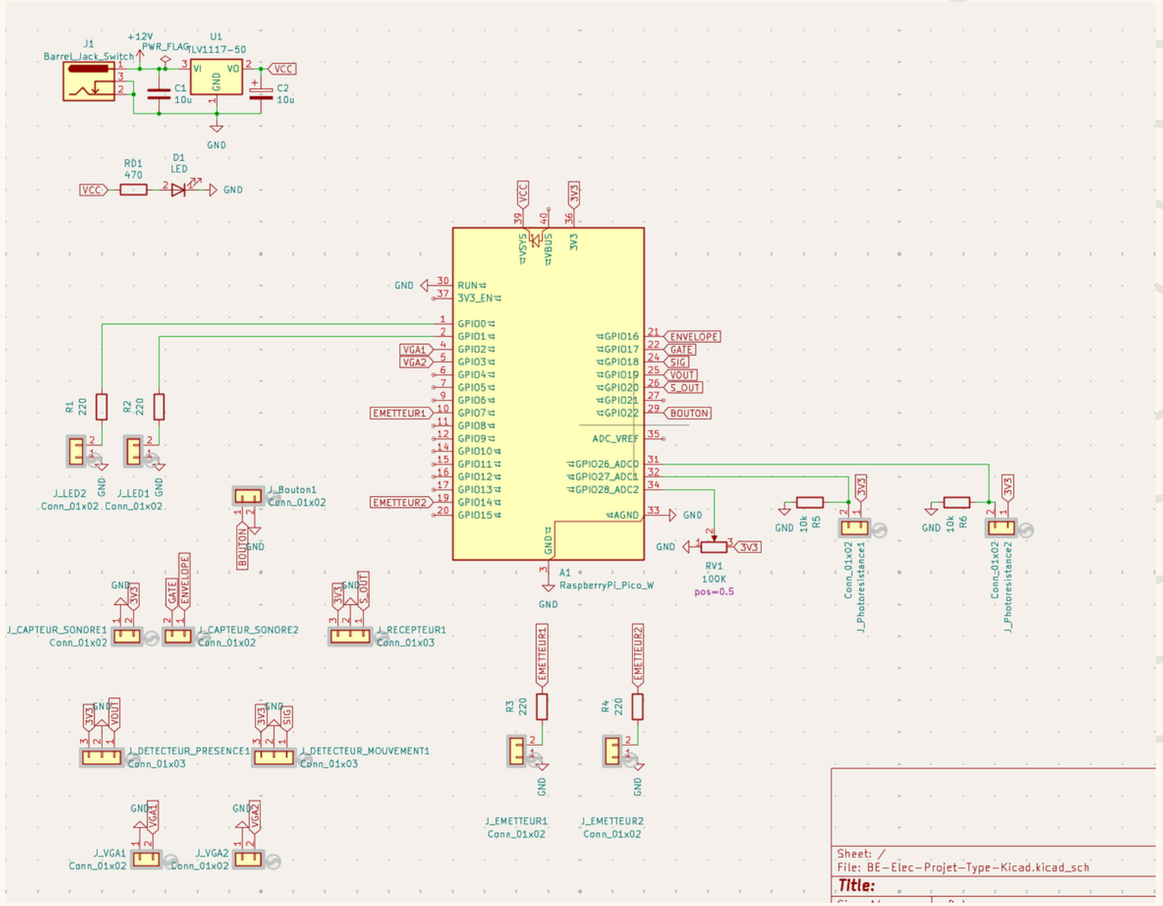


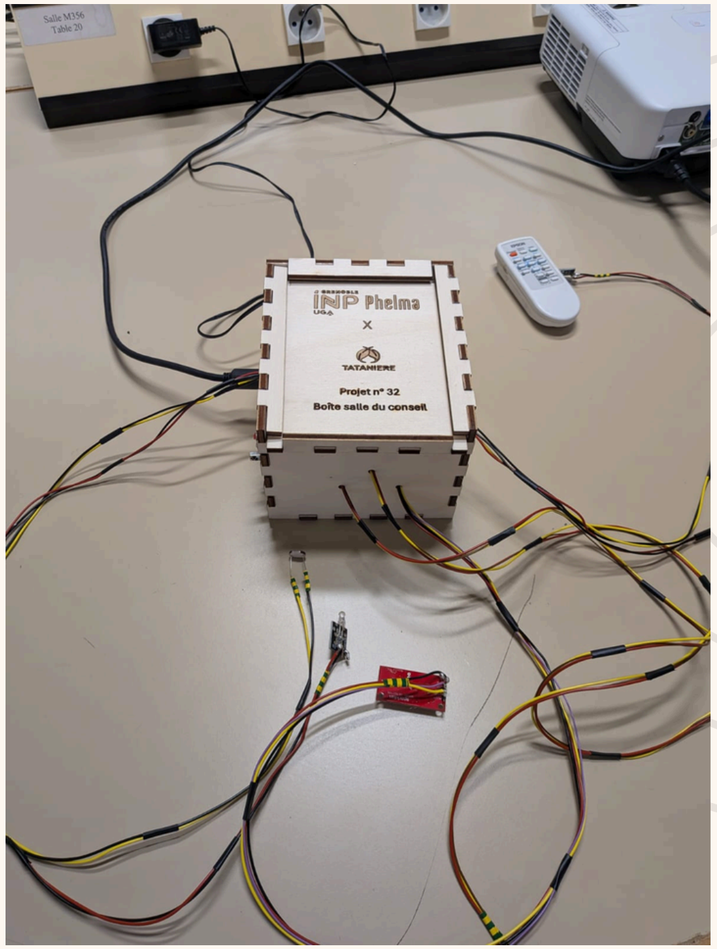
SCHÉMA KICAD

RÉSULTAT FINAL



DIMENSIONS (CM)

14
X
15.5
X
10



TESTS ET RESULTATS

OBJECTIFS :

- LED 1 ALLUMÉE SI PROJECTEUR ALLUMÉ
- LED 2 ALLUMÉE SI ON DÉTECTE LA PRÉSENCE DE QUELQU'UN (DÉTECTEUR DE PRÉSENCE, DÉTECTEUR SONORE, DÉTECTEUR DE MOUVEMENT)
- EXTINCTION AUTOMATIQUE DU PROJETEUR APRÈS UNE CERTAINE DURÉE
- EXTINCTION DU PROJECTEUR EN CAS DE SURCHAUFFE



TEST DU CAPTEUR DE PRÉSENCE EN CONDITION RÉELLE

L'IA DANS NOTRE PROJET

QUELLE IA ?

Gemini

POURQUOI CELLE-LÀ ?

- Accès gratuit à Gemini Pro
- Même fonctionnalités que d'autres IA existantes (Claude, ChatGPT, Le Chat Mistral, Deepseek)
- Pas de nécessité d'outils plus avancés

L'IA DANS NOTRE PROJET

POURQUOI L'UTILISE-T-ON ?

- Débogage du code Arduino
- Idées supplémentaires de conditions sur l'envoi ou non du signal
- Recherche d'informations dans la doc du vidéoprojecteur et dans les docs du testeur / bibliothèques arduino

L'IA DANS NOTRE PROJET

COMMENT ?

Contexte du projet :

- On explique le but du projet ainsi que le cahier des charges global

Problème que l'on rencontre :

- On explique la démarche que l'on a suivi, ce que l'on souhaite faire, et le problème qui se pose à nous, ainsi que la question que l'on se pose

Idées de pistes à étudier :

- On donne des idées de pistes à étudier, ainsi que des débuts d'éléments de réponses, en laissant la porte ouverte pour des nouvelles réponses

L'IA DANS NOTRE PROJET

EXEMPLE :

ETAT DU VIDEOPROJECTEUR

Le but du projet est de créer un boitier capable d'éteindre sous certaines conditions deux vidéoprojecteurs placés dans une salle des conseils. En effet après les réunions les vidéoprojecteurs restent parfois allumés. Cela pose des problèmes au niveau énergétique et aussi au niveau de la lentille du vidéoprojecteur qui peut griller à cause de la surchauffe.

On souhaite connaître l'état du vidéoprojecteur à un instant t, et on ne sait pas trop comment le savoir. En effet il semble compliqué de coller un capteur qui vérifie l'état grâce aux LEDS du vidéoprojecteur. Je joins en PDF la doc de notre modèle qui est assez vétuste. As-tu des solutions en réserve pour répondre à notre besoin ?

Tu peux rechercher dans la doc des éléments de réponse, et également sur ce qui est sur le WEB pour savoir si d'autres méthodes existent. On avait pensé à connaître l'état du système via le port USB avec une possible fonctionnalité du système mais nous n'avons pas réussi à trouver d'éléments de doc pour réaliser cela.

L'IA DANS NOTRE PROJET

IMPACT ECOLOGIQUE (SELON DONNÉES DE GOOGLE)

- **Energie par requête textuelle médiane :**

0.24 Wh → 0.03 Wh pour une recherche classique sur le web (8x supérieur)

Équivalent au fonctionnement d'un micro-onde pendant une seconde

- **Carbone :**

0.03g de CO_2

- **Eau :**

0.26 mL

Équivalent à 5 gouttes d'eau douce

MERCI !

Présenté par le groupe 32

ANNEXE : CODE COMPLET (1/3)

```
void callback_voie_1() {
    uint32_t maintenant = micros();
    if (digitalRead(ENTREES[1]) == HIGH) {
        debut_impulsion[1] = maintenant;
    } else {
        uint32_t duree = maintenant - debut_impulsion[1];
        if (duree >= PWM_MIN && duree <= PWM_MAX)
            valeurs_pwm[1] = (int)duree;
    }
}

void callback_voie_2() {
    uint32_t maintenant = micros();
    if (digitalRead(ENTREES[2]) == HIGH) {
        debut_impulsion[2] = maintenant;
    } else {
        uint32_t duree = maintenant - debut_impulsion[2];
        if (duree >= PWM_MIN && duree <= PWM_MAX)
            valeurs_pwm[2] = (int)duree;
    }
}

int pwm_vers_sbus(int pwm) {
    return SBUS_MIN + (pwm - PWM_MIN) * (SBUS_MAX - SBUS_MIN) / (PWM_MAX - PWM_MIN);
}
```

ANNEXE : CODE COMPLET (2/3)

```
void construire_trame_sbus(uint8_t *trame, int *voies)
{
    memset(trame, 0, SBUS_FRAME_LEN);
    trame[0]  = 0x0F;
    trame[23] = 0x00;
    trame[24] = 0x00;

    int bit = 0;
    for (int i = 0; i < NB_VOIES_SBUS; i++) {
        int valeur = voies[i];
        for (int b = 0; b < 11; b++) {
            if (valeur & (1 << b))
                trame[1 + bit / 8] |= (1 << (bit % 8));
            bit++;
        }
    }

    for (int i = 0; i < SBUS_FRAME_LEN; i++)
        trame[i] = ~trame[i];
}
```

```
void setup()
{
    Serial.begin(115200);

    // Entrées PWM
    pinMode(ENTREES[0], INPUT_PULLDOWN);
    attachInterrupt(digitalPinToInterrupt(ENTREES[0]), callback_voie_0, CHANGE);

    pinMode(ENTREES[1], INPUT_PULLDOWN);
    attachInterrupt(digitalPinToInterrupt(ENTREES[1]), callback_voie_1, CHANGE);

    pinMode(ENTREES[2], INPUT_PULLDOWN);
    attachInterrupt(digitalPinToInterrupt(ENTREES[2]), callback_voie_2, CHANGE);

    // Sorties PWM
    for (int i = 0; i < NB_VOIES_ENTREE; i++) {
        servos[i].attach(SORTIES[i], PWM_MIN, PWM_MAX);
        servos[i].writeMicroseconds(PWM_MID);
    }

    // Sortie S.Bus sur GPIO 0
    Serial1.begin(100000, SERIAL_8E2);

    Serial.println("Système démarré : 3 servos + S.Bus.");
}
```

ANNEXE : CODE COMPLET (3/3)

```
void loop()
{
    for (int i = 0; i < NB_VOIES_ENTREE; i++)
        servos[i].writeMicroseconds(valeurs_pwm[i]);

    int voies[NB_VOIES_SBUS];
    for (int i = 0; i < NB_VOIES_SBUS; i++) {
        if (i < NB_VOIES_ENTREE)
            voies[i] = pwm_vers_sbus(valeurs_pwm[i]);
        else
            voies[i] = SBUS_MID;
    }

    uint8_t trame[SBUS_FRAME_LEN];
    construire_trame_sbus(trame, voies);
    Serial1.write(trame, SBUS_FRAME_LEN);

    for (int i = 0; i < NB_VOIES_ENTREE; i++) {
        Serial.print("V");
        Serial.print(i + 1);
        Serial.print("=");
        Serial.print(valeurs_pwm[i]);
        Serial.print("µs  ");
    }
    Serial.print("|  Trame: ");
    for (int i = 0; i < SBUS_FRAME_LEN; i++) {
        if (trame[i] < 0x10) Serial.print("0");
        Serial.print(trame[i], HEX);
        Serial.print(" ");
    }
    Serial.println();
```

ANNEXE : CONSOLE ARDUINO

Output Serial Monitor ×

Message (Enter to send message to 'Raspberry Pi Pico' on 'COM4')

```
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1101µs  V3=1101µs  |  Trame: F0 B0 76 B5 AB 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1101µs  V3=1101µs  |  Trame: F0 B0 76 B5 AB 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1101µs  |  Trame: F0 B0 86 B5 AB 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1101µs  |  Trame: F0 B0 86 B5 AB 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1100µs  V3=1100µs  |  Trame: F0 B0 86 35 AC 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
V1=1100µs  V2=1101µs  V3=1101µs  |  Trame: F0 B0 76 B5 AB 01 08 40 00 02 10 80 00 04 20 00 01 08 40 00 02 10 80 FF FF
```

ANNEXE : SOURCES PARTIE 1

Documentation Sbus : https://opentx-doc.fr/wiki/pwm-cppm-s-bus-kesako-ee
Forum arduino : https://forum.arduino.cc/t/generate-s-bus-packets-with-arduino/950346/6
Doc Testeur : https://www.r-models.eu/accessoires/967-testeur-de-servo-st-v3-protronik-de-a2pro.html

ANNEXE : CODE COMPLET (1/3)

```python
import machine
import time
import utime

# ============================================================
# CONFIGURATION GÉNÉRALE
# ============================================================
ACTIVER_SON = True
ACTIVER_MOUVEMENT = True
ACTIVER_PRESENCE = True
DELAI_INACTIVITE_SEC = 5000  # 1h30 (Mettre à 10 pour les tests)

# ============================================================
# VALEURS PAR DÉFAUT (PROJECTEUR DE LA SALLE DES CONSEILS)
# ============================================================
NEC_B1 = 0x83
NEC_B2 = 0x55
NEC_B3 = 0x90
NEC_B4 = 0x6F

# ============================================================
# CONFIGURATION DES BROCHES
# ============================================================
# Entrées Numériques (Présence et État Projecteur)
capteur_son = machine.Pin(21, machine.Pin.IN, machine.Pin.PULL_DOWN)
mmwave_1 = machine.Pin(16, machine.Pin.IN, machine.Pin.PULL_DOWN)
mmwave_2 = machine.Pin(17, machine.Pin.IN, machine.Pin.PULL_DOWN)
pin_etat_projecteur = machine.Pin(2, machine.Pin.IN, machine.Pin.PULL_DOWN)

# Entrées Analogiques (ADC)
ldr_1 = machine.ADC(26)         # Photorésistance 1
ldr_2 = machine.ADC(27)         # Photorésistance 2
potentiometre = machine.ADC(28) # Potentiomètre de réglage du seuil

# Entrées/Sorties Infrarouge & Apprentissage
led_presence = machine.Pin(0, machine.Pin.OUT)
bouton_apprentissage = machine.Pin(19, machine.Pin.IN, machine.Pin.PULL_UP)
recepteur_ir = machine.Pin(18, machine.Pin.IN, machine.Pin.PULL_UP)

# Broche de sortie pour la LED témoin projecteur
led_projecteur = machine.Pin(1, machine.Pin.OUT)

# Configuration de la LED IR 1 (Émetteur 1 sur GPIO 7)
led_ir = machine.PWM(machine.Pin(7))
led_ir.freq(38000)
led_ir.duty_u16(0)

# Configuration de la LED IR 2 (Émetteur 2 sur GPIO 14)
led_ir_2 = machine.PWM(machine.Pin(14))
led_ir_2.freq(38000)
led_ir_2.duty_u16(0)

# ============================================================
# UTILITAIRES & FONCTION MAP
# ============================================================
def map_valeur(valeur, in_min, in_max, out_min, out_max):
    return int((valeur - in_min) * (out_max - out_min) / (in_max - in_min) + out_min)

# ============================================================
# FONCTIONS : ÉMISSION INFRAROUGE (NEC)
# ============================================================
def mark(duree_us):
    # On active le signal à 50% sur les DEUX leds en même temps
    led_ir.duty_u16(32768)
    led_ir_2.duty_u16(32768)
    utime.sleep_us(duree_us)
    # On coupe le signal sur les deux leds
    led_ir.duty_u16(0)
    led_ir_2.duty_u16(0)

def space(duree_us):
    led_ir.duty_u16(0)
    led_ir_2.duty_u16(0)
    utime.sleep_us(duree_us)

def zero(): mark(562); space(562)
def one(): mark(562); space(1675)

def envoyer_code_nec_universel():
    global NEC_B1, NEC_B2, NEC_B3, NEC_B4
    print(f"\n[IR] Envoi du code d'extinction -> [{hex(NEC_B1)}, {hex(NEC_B2)}, {hex(NEC_B3)}, {hex(NEC_B4)}]")

    # On génère un aperçu de ce qui va être réellement envoyé dans les airs
    debug_str = ""
    for octet in [NEC_B1, NEC_B2, NEC_B3, NEC_B4]:
        for i in range(8):
            bit = (octet >> i) & 1
            debug_str += str(bit)
        debug_str += " " # Espace entre chaque octet pour la lisibilité

    # Affichage dans la console
    print(f"[BITS] Séquence générée : {debug_str}")
```

ANNEXE : CODE COMPLET (2/3)

```
    # On boucle 3 fois pour forcer le projecteur à prendre la commande
    for envoi in range(3):
        # Petit log pour suivre les 3 tirs
        print(f"   -> Tir {envoi + 1}/3...")

        mark(9000)
        space(4500)
        for octet in [NEC_B1, NEC_B2, NEC_B3, NEC_B4]:
            for i in range(8):
                if (octet >> i) & 1:
                    one()
                else:
                    zero()
        mark(562)

        # On coupe les LEDs
        led_ir.duty_u16(0)
        led_ir_2.duty_u16(0)

        # Pause obligatoire de 50ms entre deux envois NEC
        time.sleep(0.5)

# ============================================================
# FONCTION : APPRENTISSAGE INFRAROUGE (SNIFFER)
# ============================================================
def lancer_mode_apprentissage():
    global NEC_B1, NEC_B2, NEC_B3, NEC_B4
    for _ in range(3):
        led_presence.value(1); time.sleep(0.2)
        led_presence.value(0); time.sleep(0.2)

    print("\n[MODE APPRENTISSAGE ACTIF]")
    print("Pointez la télécommande vers le capteur et appuyez sur 'Power'...")

    debut_attente = time.ticks_ms()
    while recepteur_ir.value() == 1:
        if time.ticks_diff(time.ticks_ms(), debut_attente) > 10000:
            print("[TIMEOUT] Aucun signal reçu. Retour au mode normal.")
            return

    historique = []
    etat_actuel = 0
    debut_capture = time.ticks_us()
```

```
    while time.ticks_diff(time.ticks_us(), debut_capture) < 100000:
        nouvel_etat = recepteur_ir.value()
        if nouvel_etat != etat_actuel:
            historique.append(time.ticks_us())
            etat_actuel = nouvel_etat

    durees = [time.ticks_diff(historique[i], historique[i-1]) for i in range(1, len(historique))]

    if len(durees) >= 67 and durees[0] > 8000:
        bits = [1 if durees[i] > 1200 else 0 for i in range(3, 67, 2)]
        NEC_B1 = sum(bits[0+i] << i for i in range(8))
        NEC_B2 = sum(bits[8+i] << i for i in range(8))
        NEC_B3 = sum(bits[16+i] << i for i in range(8))
        NEC_B4 = sum(bits[24+i] << i for i in range(8))
        print(f"[SUCCÈS] Nouveau code enregistré : [{hex(NEC_B1)}, {hex(NEC_B2)}, {hex(NEC_B3)}, {hex(NEC_B4)}]\n")
        led_presence.value(1); time.sleep(2); led_presence.value(0)
    else:
        print("[ERREUR] Signal illisible ou non conforme au protocole NEC.\n")

# ============================================================
# BOUCLE PRINCIPALE
# ============================================================
print("--- Système de gestion complet démarré ---")

presence_globale = False
dernier_mouvement_temps = time.time()
ir_deja_envoye = False
temps_pression_bouton = 0
bouton_en_cours = False

try:
    while True:
        # 1. GESTION DU BOUTON (Appui long de 3 secondes)
        if bouton_apprentissage.value() == 0:  # Déclenchement à 0 suite au PULL_UP
            if not bouton_en_cours:
                temps_pression_bouton = time.ticks_ms()
                bouton_en_cours = True
            elif time.ticks_diff(time.ticks_ms(), temps_pression_bouton) > 3000:
                lancer_mode_apprentissage()
                bouton_en_cours = False
                while bouton_apprentissage.value() == 0:
                    time.sleep(0.1)
                dernier_mouvement_temps = time.time()
        else:
            bouton_en_cours = False
```

ANNEXE : CODE COMPLET (3/3)

```
        # 2. LECTURE DE LA LUMINOSITÉ ET DU POTENTIOMÈTRE
        val_ldr1 = ldr_1.read_u16()
        val_ldr2 = ldr_2.read_u16()
        val_pot  = potentiometre.read_u16()

        # Ajustement de la plage du seuil (ici réglable de 30000 à 65535)
        seuil_dynamique = map_valeur(val_pot, 0, 65535, 30000, 50000)

        # Sécurité : trop de lumière si l'une des deux LDR passe sous le seuil
        trop_de_lumiere = (val_ldr1 > seuil_dynamique) or (val_ldr2 > seuil_dynamique)

        # 3. LECTURE DES CAPTEURS DE PRÉSENCE
        son_actif = (capteur_son.value() == 1) and ACTIVER_SON
        mouvement_1_actif = (mmwave_1.value() == 1) and ACTIVER_MOUVEMENT
        mouvement_2_actif = (mmwave_2.value() == 1) and ACTIVER_PRESENCE

        # 4. LOGIQUE DE PRÉSENCE
        if son_actif or mouvement_1_actif or mouvement_2_actif:
            dernier_mouvement_temps = time.time()
            ir_deja_envoye = False

            if not presence_globale:
                capteurs_actifs = []
                if son_actif: capteurs_actifs.append("Capteur Sonore")
                if mouvement_1_actif: capteurs_actifs.append("Capteur mouvement")
                if mouvement_2_actif: capteurs_actifs.append("Capteur présence")

                liste_capteurs = ", ".join(capteurs_actifs)
                print(f"[PRÉSENCE] -> Personne détectée par : {liste_capteurs} !")

            presence_globale = True
            led_presence.value(1)
        else:
            if presence_globale:
                print("[PRÉSENCE] -> Salle vide. Le chronomètre démarre.")
            presence_globale = False
            led_presence.value(0)

        # 5. GESTION DE L'EXTINCTION AUTOMATIQUE OU SÉCURITÉ LUMIÈRE
        projecteur_est_allume = (pin_etat_projecteur.value() == 1)

        if projecteur_est_allume and not ir_deja_envoye:

            # Condition A : Trop de lumière dans la pièce (Extinction immédiate !)
            if trop_de_lumiere:
                print(f"\n[SÉCURITÉ TEMPÉRATURE] Température excessive (LDR1:{val_ldr1} | LDR2:{val_ldr2} | Seuil:{seuil_dynamique}).")
                envoyer_code_nec_universel()
                ir_deja_envoye = True

            # Condition B : Pas de présence et temps écoulé dépassé
            elif not presence_globale:
                temps_ecoule = time.time() - dernier_mouvement_temps
                if temps_ecoule >= DELAI_INACTIVITE_SEC:
                    print(f"\n[TIMEOUT] Plus de {DELAI_INACTIVITE_SEC}s d'inactivité.")
                    envoyer_code_nec_universel()
                    ir_deja_envoye = True

        # 6. GESTION DE LA LED TÉMOIN PROJECTEUR EN PARALLÈLE
        if pin_etat_projecteur.value() == 1:
            led_projecteur.value(1)   # Allume la LED du GPIO 1
        else:
            led_projecteur.value(0)   # Éteint la LED du GPIO 1

        time.sleep(0.1)

except KeyboardInterrupt:
    print("\nArrêt du programme.")
finally:
    # Sécurisation complète des composants à la déconnexion
    led_presence.value(0)
    led_projecteur.value(0)
    led_ir.duty_u16(0)
    led_ir_2.duty_u16(0)
```


GRENOBLE
INP
UGA
Phelma